Thesis: A Comparative Study of Large Language Models for Financial Sentiment Analysis and Their Predictive Potential for Short-Term Stock Price Movement

Component: Experiment 1 - Model C : Mistral (7B Instruct v0.3)

Description: This notebook evaluates Mistral 7B on the Financial PhraseBank dataset under zero-shot conditions. It generates sentiment predictions and computes accuracy, precision, recall, F1-score (macro-averaged), and confusion matrix.

Select T4 GPU as runtime.

```
# Step 1 — Install bitsandbytes (restart required after this)

!pip install -q -U bitsandbytes accelerate
```

Go to runtime -> restart this session again -> then run cell 1 and run cell 2

```
# Import required libraries — Experiment 1c: Mistral

import pandas as pd
import numpy as np
import torch
import warnings
import gc

from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, classification_report
)

warnings.filterwarnings("ignore")

print(f"PyTorch version : {torch.__version__}")
print(f"CUDA available  : {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU detected    : {torch.cuda.get_device_name(0)}")
```

```
PyTorch version : 2.10.0+cu128
CUDA available  : True
GPU detected    : Tesla T4
```

Add the API Key from Hugging Face using "Add New Secret" in Google Colab

```
# Connect to Hugging Face and load Financial PhraseBank

from google.colab import userdata
from huggingface_hub import login, hf_hub_download

hf_token = userdata.get("HF_TOKEN")
login(token=hf_token)

print("Hugging Face login successful.")

# Load Financial PhraseBank — full dataset
file_path = hf_hub_download(
    repo_id="takala/financial_phrasebank",
    filename="sentences_50agree/train-00000-of-00001.parquet",
    repo_type="dataset",
    revision="0dd3028d70cbd18ded8887e65e83343b03a50482",
    token=hf_token
)

df_fpb = pd.read_parquet(file_path)

label_map = {0: "negative", 1: "neutral", 2: "positive"}
df_fpb["sentiment"] = df_fpb["label"].map(label_map)

true_labels = df_fpb["sentiment"].tolist()
texts = df_fpb["sentence"].tolist()

print(f"\nFinancial PhraseBank loaded.")
print(f"Total sentences : {len(df_fpb)}")
print(f"Label distribution:")
print(df_fpb["sentiment"].value_counts())
```

```
Hugging Face login successful.
```

```
sentences_50agree/train-00000-of-00001.p(…): 100%  390k/390k [00:00<00:00, 1.12MB/s]
```

```
Financial PhraseBank loaded.
Total sentences : 4846
Label distribution:
sentiment
neutral     2879
positive    1363
negative     604
Name: count, dtype: int64
```

```
# Load Mistral 7B Instruct v0.3

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True
)

print("Loading Mistral 7B Instruct v0.3...")

mistral_tokenizer = AutoTokenizer.from_pretrained(
    "mistralai/Mistral-7B-Instruct-v0.3",
    token=hf_token
)

mistral_model = AutoModelForCausalLM.from_pretrained(
    "mistralai/Mistral-7B-Instruct-v0.3",
    quantization_config=bnb_config,
    device_map="auto",
    token=hf_token
)

print("Mistral 7B loaded successfully.")
```

```
Loading Mistral 7B Instruct v0.3...
```

```
config.json: 100%  601/601 [00:00<00:00, 15.6kB/s]
```

```
tokenizer_config.json:  141k/? [00:00<00:00, 2.62MB/s]
```

```
tokenizer.json:  1.96M/? [00:00<00:00, 19.4MB/s]
```

```
tokenizer.model: 100%  587k/587k [00:00<00:00, 2.83MB/s]
```

```
special_tokens_map.json: 100%  414/414 [00:00<00:00, 11.3kB/s]
```

```
model.safetensors.index.json:  23.9k/? [00:00<00:00, 1.08MB/s]
```

```
Download complete: 100%  14.5G/14.5G [07:24<00:00, 135MB/s]
```

```
Fetching 3 files: 100%  3/3 [07:24<00:00, 183.18s/it]
```

```
Loading weights: 100%  291/291 [01:00<00:00, 189.41it/s, Materializing param=model.norm.weight]
```

```
generation_config.json: 100%  116/116 [00:00<00:00, 14.4kB/s]
```

```
Mistral 7B loaded successfully.
```

```
# Define Mistral classifier and run on full dataset

import logging
logging.getLogger("transformers").setLevel(logging.ERROR)

def clean_label(label):
    label = label.strip().lower()
    if "positive" in label:
        return "positive"
    elif "negative" in label:
        return "negative"
    elif "neutral" in label:
        return "neutral"
    else:
        return "neutral"

def classify_mistral(text):
    prompt = f"""You are a financial sentiment classifier.

Classify the sentiment of this financial text into exactly one word: positive, negative, or neutral.

Text: {text}

Sentiment:"""
```

```
    inputs = mistral_tokenizer(
        prompt,
        return_tensors="pt"
    ).to(mistral_model.device)

    with torch.no_grad():
        outputs = mistral_model.generate(
            **inputs,
            max_new_tokens=5,
            do_sample=False,
            pad_token_id=mistral_tokenizer.eos_token_id
        )

    input_length = inputs["input_ids"].shape[1]
    generated = mistral_tokenizer.decode(
        outputs[0][input_length:],
        skip_special_tokens=True
    )
    return clean_label(generated)

# Run on full dataset
mistral_preds = []
total = len(texts)

print(f"Running Mistral on {total} sentences...")

for i, text in enumerate(texts):
    label = classify_mistral(text)
    mistral_preds.append(label)

    if (i + 1) % 100 == 0:
        print(f"  Progress: {i+1}/{total}")

print(f"\nDone. Total predictions: {len(mistral_preds)}")
print(f"\nPrediction distribution:")
print(pd.Series(mistral_preds).value_counts())
```

```
Running Mistral on 4846 sentences...
  Progress: 100/4846
  Progress: 200/4846
  Progress: 300/4846
  Progress: 400/4846
  Progress: 500/4846
  Progress: 600/4846
  Progress: 700/4846
  Progress: 800/4846
  Progress: 900/4846
  Progress: 1000/4846
  Progress: 1100/4846
  Progress: 1200/4846
  Progress: 1300/4846
  Progress: 1400/4846
  Progress: 1500/4846
  Progress: 1600/4846
  Progress: 1700/4846
  Progress: 1800/4846
  Progress: 1900/4846
  Progress: 2000/4846
  Progress: 2100/4846
  Progress: 2200/4846
  Progress: 2300/4846
  Progress: 2400/4846
  Progress: 2500/4846
  Progress: 2600/4846
  Progress: 2700/4846
  Progress: 2800/4846
  Progress: 2900/4846
  Progress: 3000/4846
  Progress: 3100/4846
  Progress: 3200/4846
  Progress: 3300/4846
  Progress: 3400/4846
  Progress: 3500/4846
  Progress: 3600/4846
  Progress: 3700/4846
  Progress: 3800/4846
  Progress: 3900/4846
  Progress: 4000/4846
  Progress: 4100/4846
  Progress: 4200/4846
  Progress: 4300/4846
  Progress: 4400/4846
  Progress: 4500/4846
  Progress: 4600/4846
  Progress: 4700/4846
  Progress: 4800/4846
```

```
Done. Total predictions: 4846

Prediction distribution:
neutral     2887
positive    1414
negative     545
Name: count, dtype: int64
```

```
# Evaluate Mistral performance on full dataset

labels_order = ["positive", "negative", "neutral"]

acc  = accuracy_score(true_labels, mistral_preds)
prec = precision_score(true_labels, mistral_preds, average="macro", labels=labels_order)
rec  = recall_score(true_labels, mistral_preds, average="macro", labels=labels_order)
f1   = f1_score(true_labels, mistral_preds, average="macro", labels=labels_order)

print("=" * 45)
print("Mistral 7B — Experiment 1 Results")
print("=" * 45)
print(f"  Accuracy  : {acc:.4f}  ({acc*100:.2f}%)")
print(f"  Precision : {prec:.4f}")
print(f"  Recall    : {rec:.4f}")
print(f"  F1-Score  : {f1:.4f}")
print("=" * 45)

print("\nDetailed Classification Report:")
print(classification_report(true_labels, mistral_preds, labels=labels_order))
```

```
=============================================
Mistral 7B — Experiment 1 Results
=============================================
  Accuracy  : 0.7961  (79.61%)
  Precision : 0.8039
  Recall    : 0.7845
  F1-Score  : 0.7934
=============================================

Detailed Classification Report:
              precision    recall  f1-score   support

    positive       0.70      0.72      0.71      1363
    negative       0.89      0.80      0.84       604
     neutral       0.83      0.83      0.83      2879

    accuracy                           0.80      4846
   macro avg       0.80      0.78      0.79      4846
weighted avg       0.80      0.80      0.80      4846
```

```
# Save Mistral results

mistral_scores = {
    "model": "Mistral 7B",
    "accuracy": round(acc, 4),
    "precision": round(prec, 4),
    "recall": round(rec, 4),
    "f1_score": round(f1, 4)
}

df_fpb["mistral_pred"] = mistral_preds

print("Mistral 7B — Results Summary")
print(pd.DataFrame([mistral_scores]))
```

```
Mistral 7B — Results Summary
        model  accuracy  precision  recall  f1_score
0  Mistral 7B    0.7961     0.8039  0.7845    0.7934
```

```
# Save Mistral predictions to Google Drive

from google.colab import drive
drive.mount("/content/drive")

import os
os.makedirs("/content/drive/MyDrive/Thesis_Data", exist_ok=True)

df_fpb.to_csv("/content/drive/MyDrive/Thesis_Data/exp1_mistral_preds.csv", index=False)

print("Mistral predictions saved to Google Drive.")
```

```
Mounted at /content/drive
Mistral predictions saved to Google Drive.
```

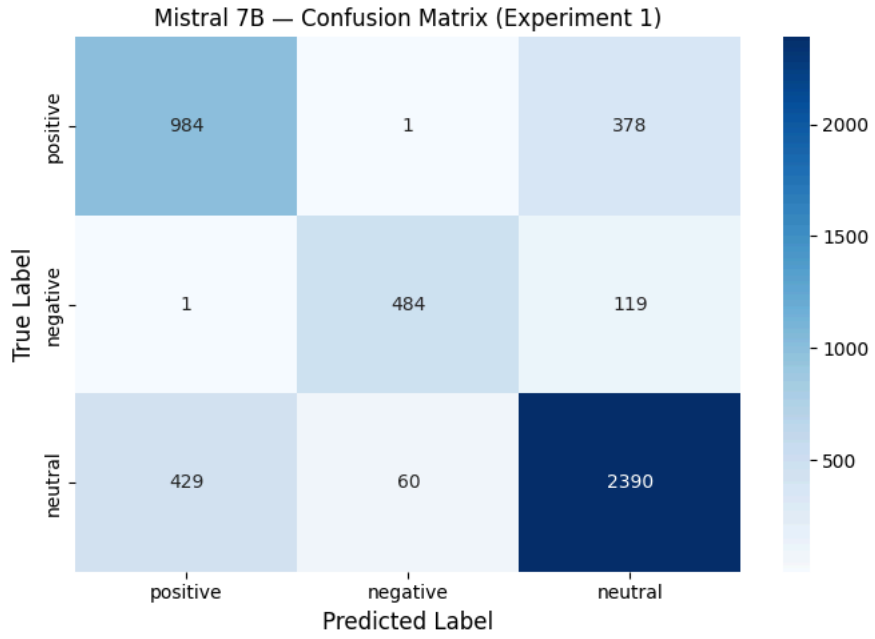
```
# Confusion Matrix — Mistral 7B Experiment 1

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

labels_order = ["positive", "negative", "neutral"]

cm = confusion_matrix(true_labels, mistral_preds, labels=labels_order)

plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=labels_order,
            yticklabels=labels_order)
plt.xlabel("Predicted Label", fontsize=12)
plt.ylabel("True Label", fontsize=12)
plt.title("Mistral 7B — Confusion Matrix (Experiment 1)", fontsize=12)
plt.tight_layout()
plt.savefig("/content/drive/MyDrive/Thesis_Data/fig_cm_mistral.png",
            dpi=300, bbox_inches="tight")
plt.show()
print("Confusion matrix saved to Google Drive.")
```




```
Confusion matrix saved to Google Drive.
```